

第15讲

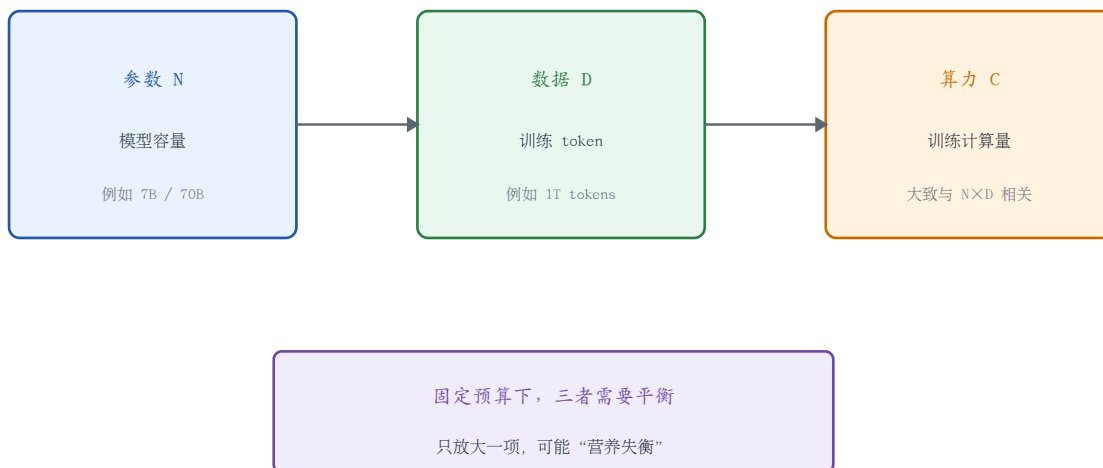
Scaling Law：为什么模型越做越大？

参数、训练数据与算力为什么必须一起扩展

本讲目标

学完后，你应该能解释：参数 N 、训练 token D 、训练算力 C 分别是什么；为什么模型更大通常有收益但不是无条件成立；什么叫 undertrained；compute-optimal training 在优化什么；Chinchilla 结论应该怎样正确理解；为什么“涌现能力”和“参数量排名”都需要谨慎解读。

Scaling 的三个核心量：参数 N 、数据 D 、训练算力 C



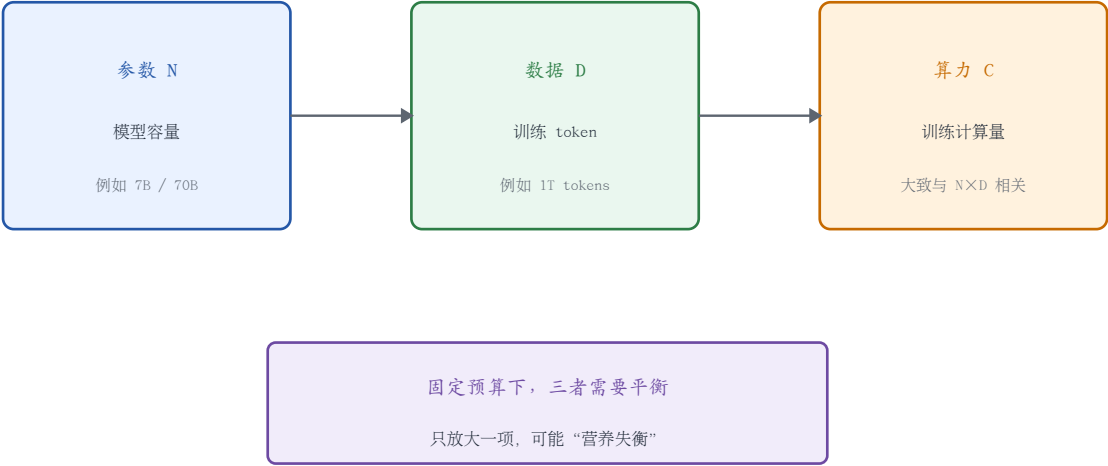
先记住一句

Scaling Law 不是“参数越多越强”的口号，而是在研究：当模型、数据和计算量改变时，训练损失和能力怎样以相对可预测的方式变化。

1. Scaling 到底是什么意思？

在大语言模型里，Scaling 通常指把某些资源维度系统地放大，例如参数量、训练 token、训练计算量、上下文长度或推理时计算量。最经典的 Scaling Law 讨论主要围绕前三个量：模型参数 N、数据量 D、计算量 C。

Scaling 的三个核心量：参数 N、数据 D、训练算力 C



不要混淆

“一个模型有 70B 参数”只告诉你模型容量的一部分信息。它没有告诉你训练过多少 token、数据质量如何、用了多少算力、架构是什么。

2. 参数 N：参数多到底意味着什么？

参数是训练过程中可更新的数值，例如 Attention 和 FFN 里的权重矩阵。参数更多，通常意味着模型拥有更大的函数表示容量，可以拟合更复杂的统计规律。

说法	是否准确	原因
“参数越多，容量通常越大”	基本准确	可表示的函数空间更大
“参数越多，一定更聪明”	不准确	还受训练数据、优化和算力限制
“参数就是知识条目数量”	错误	参数是分布式数值表示
“同参数量模型能力必然一样”	错误	架构、数据和训练方法均不同

类比

可以把参数量类比成“脑容量/可调旋钮数量”，但旋钮多不代表已经被调到好位置。训练才负责把参数调整成有用结构。

3. 数据 D：不是文档数量，而是训练 token 数量

Scaling Law 中的数据量 D 通常更接近“模型实际处理了多少训练 token”，而不是硬盘里有多少 GB 文件。一个文档可能被切成很多 token，也可能因为过滤、去重或采样根本没有进入训练 batch。

- D 反映模型获得了多少训练信号；
- 相同 token 数的数据，质量和多样性仍可能非常不同；
- 同一数据被重复训练多个 epoch，也会增加模型实际看到的 token 数；
- “unique tokens”和“training tokens”有时需要区分。

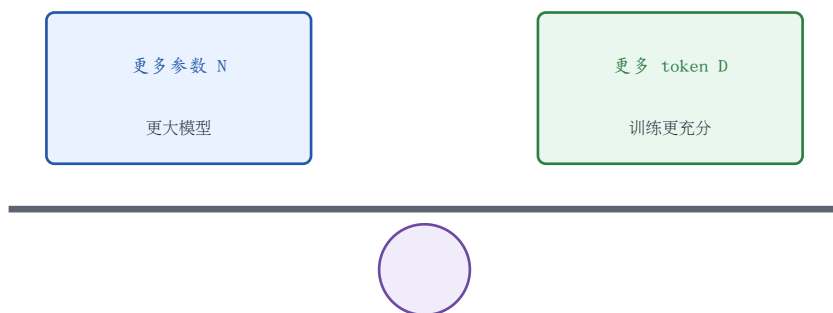
与第14讲连接

第14讲讲的是“token 从哪里来、质量如何”；本讲讲的是“在一定资源下，到底让模型吃多少 token 才划算”。

4. 算力 C：为什么粗略与 $N \times D$ 成正比？

对 Dense Transformer，训练每个 token 都需要大量矩阵乘法。模型参数越多，每个 token 的前向和反向计算通常越贵；训练 token 越多，又意味着要重复更多次计算。因此训练计算量会同时随 N 和 D 增长。

固定训练算力：参数 N 与训练 token D 的“跷跷板”



粗略地说，Dense Transformer 预训练 FLOPs 常用 $C \approx 6ND$ 做数量级估算。

这是工程估算，不是精确物理定律；架构、序列长度、优化器和实现都会改变实际成本。

工程上常见一个数量级估算：

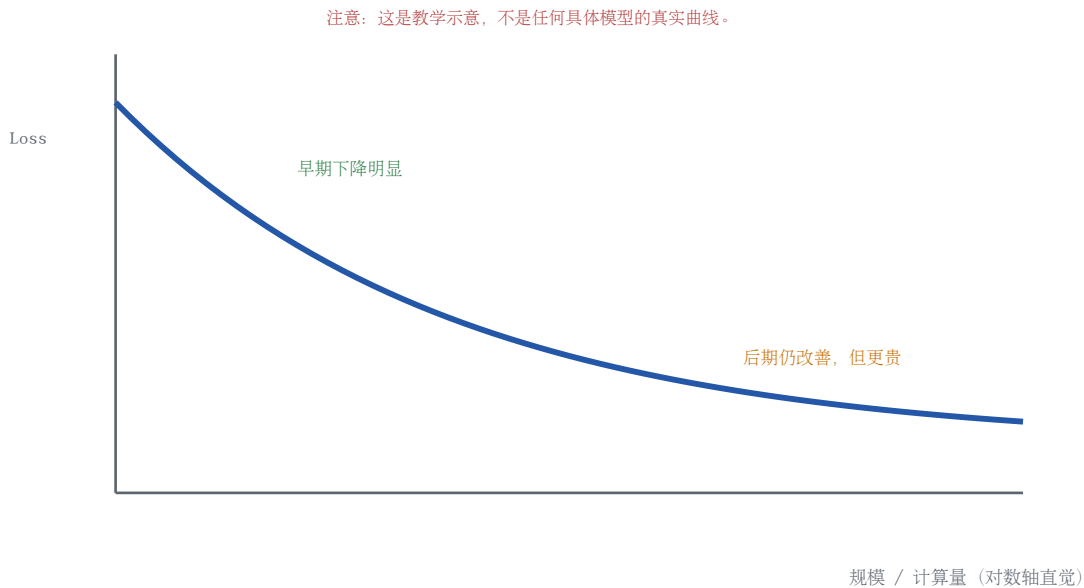
$$C \approx 6 \times N \times D \text{ FLOPs}$$

重要限制

这里的 6 是常用粗略估算系数，不是精确常数。实际训练成本还取决于 Attention 长度、激活重计算、优化器、MoE、硬件利用率和实现。

5. 什么是 Scaling Law?

典型 Scaling Law 直觉：规模增大，Loss 下降，但边际收益递减



经验上，在一定范围内，当模型规模、数据量或训练计算量增加时，语言模型的验证 loss 往往呈现较平滑、可拟合的幂律趋势。常见抽象写法类似：

$$L(x) \approx L_{\infty} + A \cdot x^{-\alpha}$$

这里 L_{∞} 可以理解成不可约部分， A 和 α 是通过实验拟合得到的系数。重点不在记公式，而在理解：收益通常持续存在，但边际收益递减。

科研价值

如果趋势足够稳定，小规模实验就可以帮助估计更大规模训练会发生什么，从而避免完全盲目地烧算力。

6. 为什么“边际收益递减”非常重要？

假设从 1B 增大到 10B 带来很明显的 loss 改善，再从 10B 增大到 100B 仍可能改善，但改善幅度未必按 10 倍同步增长，而成本却可能大幅上升。

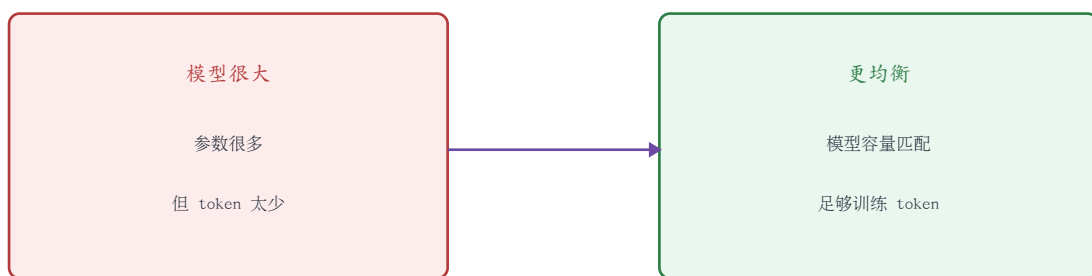
规模变化	可能的直觉
小模型 -> 中模型	能力/损失改善较明显
中模型 -> 大模型	继续改善，但成本显著增加
极大模型继续扩展	仍可能获益，但每一点收益都更贵

这不是说“不要做大模型”

而是说资源分配必须考虑收益曲线。工业界关心的是“多花 10 倍算力，到底换来多少可用能力”。

7. 为什么“大模型 + 太少 token”会 Undertrained?

为什么“大模型 + 太少数据”可能是 Undertrained?



固定训练算力下：参数多一点，就意味着通常能处理的 token 少一点；反之亦然。

Compute-optimal scaling 研究的就是：预算固定时，N 和 D 怎么分更划算。

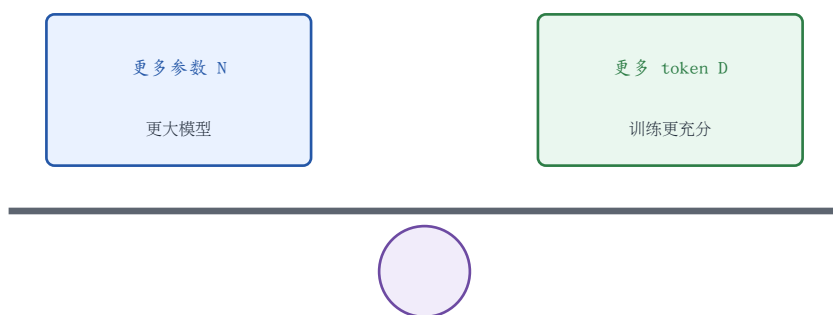
一个参数很多的模型，如果训练 token 不够，许多参数可能没有获得充分、稳定的训练信号。它虽然“容量很大”，但没有被充分训练。

通俗类比

给一个巨大的图书馆配 100 万个书架，却只放 1000 本书。书架很多并不会自动产生知识，反而意味着大量容量没有被有效利用。

8. Fixed Compute：为什么 N 和 D 像跷跷板？

固定训练算力：参数 N 与训练 token D 的“跷跷板”



粗略地说，Dense Transformer 预训练 FLOPs 常用 $C \approx 6ND$ 做数量级估算。

这是工程估算，不是精确物理定律；架构、序列长度、优化器和实现都会改变实际成本。

如果训练预算 C 固定，那么把模型 N 做大，会提高每个 token 的计算成本，通常就意味着能训练的 token D 变少；反过来，把模型做小一些，就可以在同样预算下处理更多 token。

核心问题

Compute-optimal scaling 不是问“最大模型是多少”，而是问：在固定 FLOPs 预算下， N 和 D 怎样分配才能让最终 loss 最低？

9. Kaplan 等早期 Scaling Laws 给了什么启发？

早期大规模经验研究发现，语言模型的验证损失对模型规模、数据和计算量呈现相对规律的幂律趋势。这推动了一个重要观念：大模型性能并不是完全不可预测的，扩展前可以通过较小实验做趋势估计。

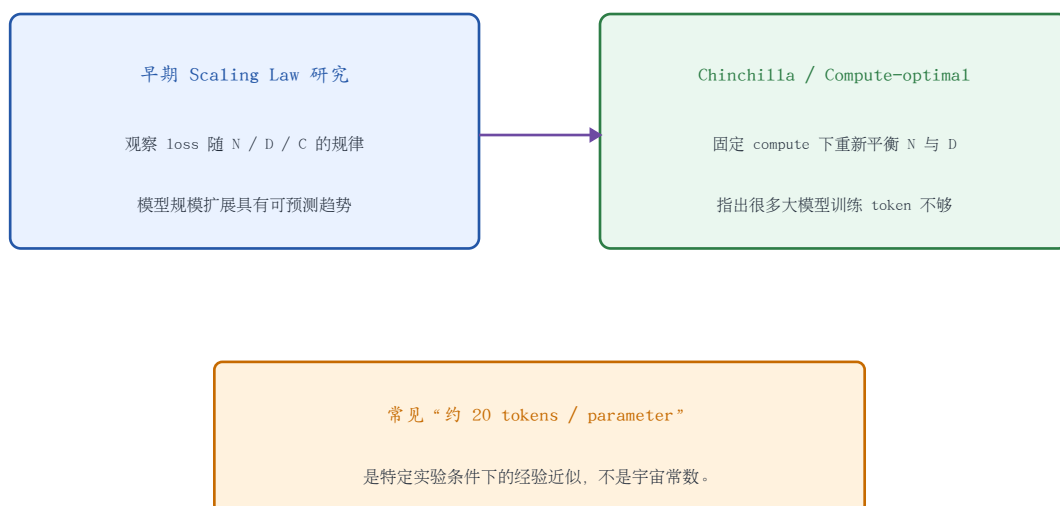
- 模型规模增大通常能降低 loss;
- 数据增多通常能降低 loss;
- 计算量增加通常能降低 loss;
- 但三者要匹配，否则会浪费资源;
- 这些规律是经验统计关系，不是数学保证。

技术史意义

Scaling Law 让“大模型研发”从纯经验工程，部分转向了可以用曲线拟合和资源预算做规划的工程科学。

10. Chinchilla：为什么它改变了“模型多大才划算”的理解？

Kaplan 路线与 Chinchilla 路线：关键问题从“能否变大”走向“怎样分配算力”



更现代的训练会综合数据质量、推理成本、上下文长度、MoE、重复 epoch 等因素。

Chinchilla 一类研究重新评估固定计算预算下的参数量与 token 配比，核心结论是：很多当时的大模型相对于自身参数量，训练 token 不够充分。与其继续堆参数，不如在同样算力下使用更小模型并训练更多 token。

不要机械背“20”

经常听到的“约 20 tokens / parameter”是对特定 Chinchilla 实验结论的简化概括。它不是适用于所有架构、数据质量、上下文长度、训练目标和部署目标的固定常数。

11. 一个简单数字例子：70B 参数意味着多少训练量？

假设仅做教学估算：一个 70B（700 亿）Dense 模型，如果按 20 tokens/parameter 的粗略比例，则训练 token 约为：

$$70B \times 20 \approx 1.4T \text{ tokens}$$

再用 $C \approx 6ND$ 做数量级估算：

$$C \approx 6 \times 70 \times 10^9 \times 1.4 \times 10^{12} \approx 5.9 \times 10^{23} \text{ FLOPs}$$

只看数量级

这个计算的目的是估算某个真实商业模型，而是建立尺度感：几十 B 参数与 T 级 token 一结合，训练计算量会迅速达到极其巨大的数量级。

12. 为什么今天很多模型会“训练得比 Chinchilla 更久”？

如果目标只是在固定一次性训练 FLOPs 下取得最低 loss, compute-optimal 是一个重要基准。但真实产品还要考虑长期推理成本。较小模型如果多训练更多 token，虽然预训练可能“过训练”，但部署时每次调用更便宜。

训练最优 $\neq$ 部署最优：推理成本会改变选择



如果模型要被调用十亿次，推理成本可能比一次预训练成本更重要。

关键转折

训练只做一次，推理可能做几十亿次。因此“训练最优”与“整个生命周期成本最优”可能不是同一个点。

13. Scaling Law 能预测 “智能” 吗？

通常 Scaling Law 首先拟合的是可连续测量的训练/验证 loss，或者某些任务指标。它不能直接把 “智能” 当作一个单一物理量来预测。

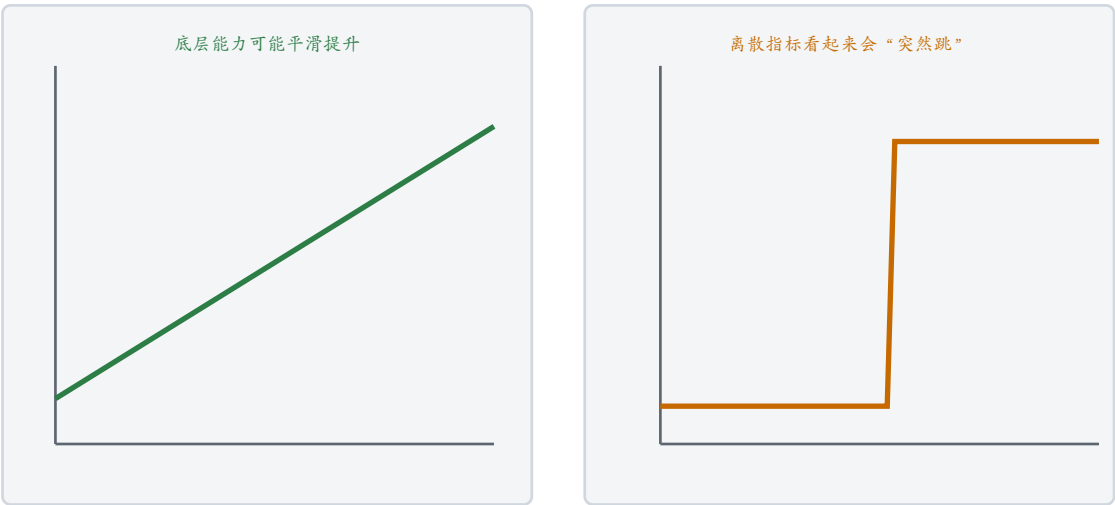
更容易预测	更难直接预测
Cross-entropy loss	复杂开放式推理能力
Perplexity 趋势	是否会突然掌握某项任务
某些 benchmark 平均分	真实世界 Agent 稳健性
训练 FLOPs 与资源需求	价值判断、创造力等抽象概念

研究习惯

以后看到 “Scaling predicts AGI” 一类说法，要先问：预测的变量到底是什么？loss、某 benchmark，还是一个未经严格定义的宏观概念？

14. Emergent Abilities：能力会不会突然 “涌现” ？

“涌现能力” 为什么既重要又容易被误解？



因此 “突然出现” 可能是真实非线性，也可能部分来自任务阈值、指标定义或采样噪声。

有些任务上，小模型几乎为 0，大到某个规模以后分数突然上升，看起来像 “能力突然出现”。这可能反映真实的非线性组合，也可能部分来自评价指标的离散阈值。

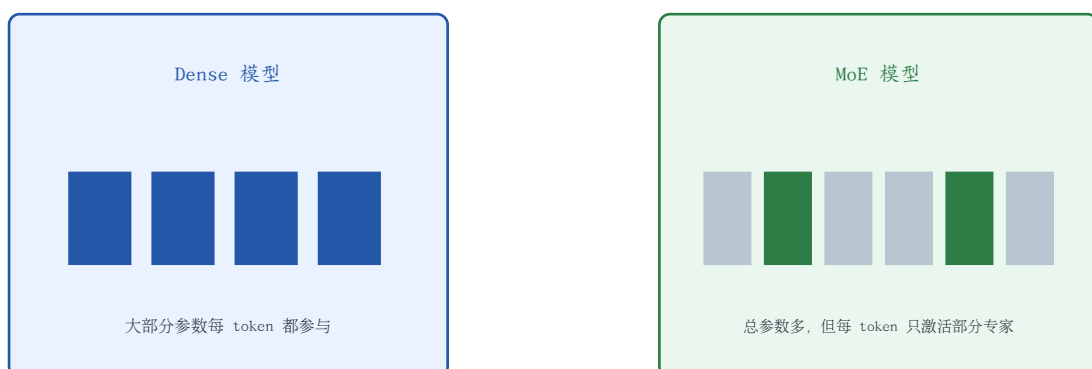
- 底层概率质量可能平滑改善；
- 但 exact match 这类指标只有 “全对/全错” ；
- 当输出刚跨过阈值时，指标会出现跳跃；
- 因此 “涌现” 是值得研究的现象，但不能仅凭一条折线就下强结论。

底层例子

模型把正确答案概率从 0.40 提高到 0.49, exact match 仍可能是 0; 从 0.49 到 0.51 后, 贪心解码突然答对, 于是宏观指标看起来像 “突然学会” 。

15. 参数量为什么越来越不能直接横向比较？

MoE 为什么让“参数量”这个数字更难直接比较？



比较模型时，应区分 total parameters、active parameters、training FLOPs 和 inference FLOPs。

Dense 模型里，大部分参数都会参与每个 token 的计算；Mixture-of-Experts (MoE) 模型则可能拥有巨大的总参数量，但每个 token 只路由到少数专家。

指标	含义
Total Parameters	模型总共存储多少参数
Active Parameters	一次 token 前向实际激活多少参数
Training FLOPs	训练过程真实计算量
Inference FLOPs	每次推理计算量
Memory Footprint	权重/KV cache/激活占用

因此

“A 是 600B，B 是 100B，所以 A 一定更大更强”在 MoE 时代尤其不可靠。必须先看 active parameters 和训练计算量。

16. Scaling 还有哪些维度？

- Context Scaling: 把上下文从 8K 扩展到 128K、1M;
- Inference-time Scaling: 回答时投入更多采样、搜索、验证或 reasoning compute;
- Data Quality Scaling: 不单纯增加 token，而是提升有效数据密度;
- Agent Scaling: 增加 rollouts、工具调用、memory 检索或多 Agent 交互;
- Hardware/System Scaling: 更多 GPU、更高并行度、更高利用率。

本讲边界

经典 Scaling Law 主要讲预训练 N/D/C。后面的推理模型课程会再单独讲 test-time compute 为什么成为新的 scaling 轴。

17. 为什么 Scaling 对独立研究者仍然重要？

你不需要训练 70B 模型，仍然应该理解 Scaling Law。因为它决定你怎样解释别人实验，也决定你设计小规模 Agent 实验时怎样避免错误比较。

- 知道为什么不能只按参数量比较模型；
- 知道较小模型可能是“训练更充分”的；
- 知道实验预算应该先做小规模趋势估计；
- 知道 API 模型之间的总参数量往往不是最重要变量；
- 知道 Agent 系统的提升可能来自底座 scaling，而不是你的协作机制。

与你的研究直接相关

以后研究多 Agent 时，必须防止“多 Agent 用了更大总 token / 更多模型调用，所以自然更强”这种混杂变量。系统级 scaling 本身就是实验变量。

18. Agent Systems 中的 Scaling：数量增加 ≠ 集体智能自动增加

多 Agent 系统也有自己的 Scaling 问题。例如从 1 个 Agent 增加到 4、16、64 个，计算成本、通信量和错误相关性都会变化。不能简单把“更多 Agent”理解成“更强”。

扩展变量	可能收益	可能代价
Agent 数量	更多并行观点	通信/协调成本
总模型调用次数	更多尝试	API 成本和延迟
讨论轮数	更多修正机会	错误可能互相强化
Memory 容量	更多历史信息	检索噪声和上下文成本
工具调用预算	更多外部证据	失败面和系统复杂度增加

科研问题

Agent Systems Science 的一个核心任务，就是把“规模 → 成本 → 性能/可靠性”的关系从经验现象变成可测量、可预测的规律。

19. 常见误区

误区	更准确的理解
“模型参数越多一定越强”	参数是一个变量，还要看 token、数据质量、算力和架构
“20 tokens/parameter 是固定定律”	是历史实验中的经验近似，实际最优值依条件变化
“Scaling Law 能精确预测所有能力”	通常更擅长预测连续 loss/指标趋势
“涌现说明能力凭空出现”	可能含真实非线性，也可能受指标阈值影响
“总参数量可以直接比较 MoE 与 Dense”	还需看 active params 与 FLOPs
“训练最优就是产品最优”	部署还要考虑长期推理成本

一句话总结

Scaling 是资源配置问题：多大的模型、多少数据、多少训练与推理计算，怎样组合才获得最有效的能力。

20. 小实验：用玩具模型直观看 N、D、C 的权衡

材料包中的 Python 脚本不会拟合真实大模型，而是构造一个明确标注为“教学用”的 toy scaling function。它做三件事：

- 固定 compute budget，枚举不同参数量 N；
- 由 $C \approx 6ND$ 反推出可用训练 token D；
- 用一个人工定义的 toy loss 函数寻找“参数太小”“参数太大但 token 不够”和较均衡区域。

运行命令：

```
python 04_scaling_law_toy_demo.py
```

实验目的

不是复现 Chinchilla 数值，而是亲眼看到：固定预算时，N 和 D 会互相挤占资源，所以最优点通常在中间，而不是无限增大某一边。

21. 本讲总结：以后看到“大模型更强”先问 8 个问题

问题	目的
参数 N 多大?	知道模型容量
训练 D 多少 token?	判断是否训练充分
训练 FLOPs C 多大?	判断真实计算投入
数据质量/配比怎样?	理解有效训练信号
Dense 还是 MoE?	区分总参数与激活参数
训练目标是什么?	知道 loss 优化什么
推理时用了多少 compute?	避免把 test-time scaling 当模型本身能力
评测是否控制总 token/API 调用?	Agent 实验中避免预算混杂

进入下一讲

第16讲将进入 Instruction Tuning: 为什么预训练 Base Model

只是“会续写”，经过指令微调后才更像一个会按照用户要求办事的助手。